

1. Что такое Spring Framework?
 2. Какова основная цель использования Spring Framework?
 3. Назовите основные модули Spring Framework.
-

1 Что такое Spring Framework?

Spring Framework — это популярный фреймворк для разработки Java-приложений, который упрощает создание корпоративных, веб- и микросервисных приложений.

Основные особенности:

- **IoC / Dependency Injection** — управление зависимостями через контейнер
- **Аспектно-ориентированное программирование (AOP)** — разделение бизнес-логики и кросс-срезовых задач
- Поддержка транзакций, интеграции с базами данных, веб-сервисов

✦ Коротко:

Spring — это инфраструктурный каркас для упрощения разработки и тестирования Java-приложений.

2 Какова основная цель использования Spring Framework?

Основная цель — **снизить сложность разработки и повысить переиспользуемость кода.**

Spring решает:

- управление зависимостями объектов (IoC)
- упрощение работы с транзакциями, базами данных и веб-сервисами
- интеграцию с различными технологиями через стандартные интерфейсы
- улучшение тестируемости через слабое связывание компонентов

Коротко для собеседования:

Spring помогает строить модульные, легко тестируемые и расширяемые приложения.

3 Основные модули Spring Framework

Модуль	Назначение
Core / Beans	IoC контейнер и управление зависимостями
Context	Расширение Core, ApplicationContext, поддержка событий и ресурсов
AOP	Аспектно-ориентированное программирование
Data Access / JDBC	Упрощённая работа с JDBC, управление транзакциями
ORM	Интеграция с JPA/Hibernate/MyBatis
Web / Web MVC	Создание веб-приложений, REST API
Messaging	Поддержка JMS, RabbitMQ, Kafka
Test	Фреймворк для unit и интеграционных тестов

 Дополнительно:

- **Spring Boot** — облегчает настройку и запуск приложений, но это отдельный проект поверх Spring Framework.

1. Объясните, как работает Inversion of Control (IoC) в Spring.
 2. Опишите процесс создания и использования Spring Data Access
 3. Objects (DAO) и какие преимущества они предлагают.
Какие особенности и преимущества предлагает Spring MVC для разработки веб-приложений?
-

1 Как работает Inversion of Control (IoC) в Spring?

Inversion of Control (IoC) — это принцип, при котором контейнер Spring управляет созданием и связыванием объектов, а не код приложения.

◆ Основная идея

- Вместо того чтобы класс создавал свои зависимости вручную:

```
class Service {  
    private Repository repo = new Repository(); // жёсткая зависимость  
}
```

- Spring создаёт объекты и внедряет их через **Dependency Injection (DI)**:

```
@Component  
class Service {  
    private final Repository repo;  
  
    @Autowired  
    public Service(Repository repo) {  
        this.repo = repo;  
    }  
}
```

Контейнер Spring:

1. Сканирует классы (Component Scan)
2. Создаёт бины
3. Внедряет зависимости автоматически

🔑 Ключевой эффект:

Класс не зависит от конкретной реализации своих зависимостей — IoC делает код более гибким и тестируемым.

2 Процесс создания и использования Spring DAO и их преимущества

◆ DAO (Data Access Object) — слой для работы с данными

Цель: абстрагировать работу с базой данных, предоставляя методы CRUD.

◆ Пример с Spring JDBC

```
@Repository
public class UserDao {

    private final JdbcTemplate jdbcTemplate;

    @Autowired
    public UserDao(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public void save(User user) {
        jdbcTemplate.update("INSERT INTO users(name, email) VALUES (?, ?)",
            user.getName(), user.getEmail());
    }

    public User findById(Long id) {
        return jdbcTemplate.queryForObject("SELECT * FROM users WHERE id =
?",
            new BeanPropertyRowMapper<>(User.class), id);
    }
}
```

◆ Преимущества DAO в Spring

1. **Абстракция доступа к данным** — код приложения не зависит от конкретного механизма хранения.
 2. **Упрощение работы с JDBC / ORM** — Spring управляет транзакциями и исключениями.
 3. **Интеграция с транзакциями** — легко использовать `@Transactional`.
 4. **Тестируемость** — можно подменить DAO на мок или stub.
-

3 Особенности и преимущества Spring MVC для веб-разработки

Spring MVC — это фреймворк для создания веб-приложений с архитектурой **Model-View-Controller**.

◆ Ключевые особенности

Особенность	Описание
DispatcherServlet	Центральный контроллер, обрабатывает все запросы
Контроллеры (@Controller / @RestController)	Обработка HTTP-запросов и возврат моделей или JSON
View Resolver	Преобразует модели в HTML, JSON или другой формат
Data Binding	Автоматическая конвертация параметров запроса в объекты
Validation	Поддержка @Valid и встроенных валидаторов
HandlerInterceptor	Фильтры/интерсепторы для логирования и авторизации

◆ Преимущества Spring MVC

1. **Разделение логики (MVC)** — легко поддерживать и тестировать
 2. **Интеграция с Spring IoC** — контроллеры и сервисы автоматически внедряются
 3. **REST API готово “из коробки”** — через @RestController и @RequestMapping
 4. **Масштабируемость** — поддержка модулей, фильтров, Interceptors
 5. **Широкая экосистема** — Spring Security, Spring Data, Spring Boot
-

◆ Пример простого контроллера

```
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserDao userDao;

    @Autowired
    public UserController(UserDao userDao) {
        this.userDao = userDao;
    }

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userDao.findById(id);
    }
}
```

1. Опишите ситуацию, когда использование Spring может быть нецелесообразно/
2. Какие практические примеры могут показать, что выбор Spring зависит от требований к проекту, включая его масштабируемость, надежность и поддержку различных форматов данных?
3. Какие современные тенденции в разработке веб-приложений и технологиях могут влиять на выбор Spring и как они могут быть адаптированы для современных приложений?

1 Когда использование Spring может быть нецелесообразно

Хотя Spring мощный и гибкий, не во всех проектах его использование оправдано:

◆ Малые проекты / простые сервисы

- Один-два класса, минимальная логика
- Сборка “чистого” Java + небольшой фреймворк (или без него) проще
- Spring добавит **overhead**: зависимостями, конфигурацией, временем старта

◆ Критические по скорости старта приложения

- Spring IoC контейнер создаёт много бинов на старте
- Для микросервисов с быстрым cold-start (например, AWS Lambda) иногда лучше **Micronaut** или **Quarkus**

◆ Сильно ограниченные ресурсы

- Spring потребляет больше памяти из-за контейнера и прокси-бинов
- Для embedded-устройств или серверов с ограниченной RAM может быть тяжело

2 Практические примеры, когда выбор Spring зависит от требований

◆ Масштабируемость

- Проект: микросервисная архитектура с десятками сервисов
- Spring Boot + Spring Cloud → упрощает конфигурацию, интеграцию, service discovery, load balancing
- Без Spring → нужно вручную управлять зависимостями, конфигурацией, интеграциями

◆ Надежность и тестируемость

- Enterprise-приложение с сложной бизнес-логикой и транзакциями
- Spring поддерживает @Transactional, интеграцию с JPA/Hibernate → меньше ошибок
- Без Spring → сложнее поддерживать консистентность данных

◆ Работа с различными форматами данных

- REST + JSON (@RestController)
- SOAP (spring-ws)
- XML, CSV, Kafka, RabbitMQ
- Spring интегрирует всё через стандартизованные API, упрощая расширяемость

3 Современные тенденции веб-разработки, влияющие на выбор Spring

◆ Микросервисы

- Spring Boot + Spring Cloud → быстрое создание REST/HTTP сервисов
- Поддержка circuit breaker, service discovery, config server

◆ Реактивное программирование

- WebFlux для асинхронных, масштабируемых приложений
- Позволяет обрабатывать тысячи соединений с меньшими ресурсами

◆ Serverless / Cloud-native

- Spring поддерживает интеграцию с AWS Lambda, Kubernetes
- Можно адаптировать через Spring Cloud Function

◆ Контейнеризация и DevOps

- Spring Boot создаёт “fat jar”, легко контейнеризировать через Docker
- Интеграция с CI/CD пайплайнами, мониторингом и логированием

◆ Поддержка современных форматов данных

- JSON, XML, Protobuf, Avro
- Интеграция с базами NoSQL (MongoDB, Cassandra) и реляционными
- Spring Data упрощает работу через единый интерфейс

1. Для чего используется аннотация `@Autowired` в Spring?
 2. Какими способами можно определить нужный класс при внедрении интерфейса?
 3. Какие реализации IoC вы можете перечислить?
-

1 Для чего используется аннотация `@Autowired` в Spring?

`@Autowired` — это механизм внедрения зависимостей (Dependency Injection, DI).

◆ Основное назначение:

- Сообщает Spring, что зависимость должна быть внедрена автоматически контейнером IoC
- Позволяет не создавать объект вручную, Spring сам найдет бин и подставит его

◆ Примеры использования

Через конструктор (рекомендуемый способ)

```
@Component
class UserService {

    private final UserRepository userRepository;

    @Autowired
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

Через поле (менее предпочтительно)

```
@Component
class UserService {

    @Autowired
    private UserRepository userRepository;
}
```

Через сеттер

```
@Component
class UserService {

    private UserRepository userRepository;

    @Autowired
    public void setUserRepository(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

✦ Рекомендуют **конструкторный DI** для иммутабельности и тестируемости.

2 Какими способами можно определить нужный класс при внедрении интерфейса?

Если несколько реализаций одного интерфейса, Spring может не знать, какой бин выбрать. Есть несколько способов:

◆ 1. @Qualifier

```
@Component
class Service {

    private final PaymentProcessor processor;

    @Autowired
    public Service(@Qualifier("paypalProcessor") PaymentProcessor processor)
    {
        this.processor = processor;
    }
}
```

◆ 2. @Primary

```
@Component
@Primary
class PaypalProcessor implements PaymentProcessor { ... }

@Component
class StripeProcessor implements PaymentProcessor { ... }

// Внедрение без Qualifier возьмёт PaypalProcessor
@Autowired
private PaymentProcessor processor;
```

◆ 3. Имя поля/параметра

- Если имя поля совпадает с именем бина, Spring может использовать его автоматически (но это менее надёжно)

◆ 4. Профили (@Profile)

- Разные реализации для dev / prod

```
@Component
@Profile("dev")
class DevPaymentProcessor implements PaymentProcessor { ... }

@Component
@Profile("prod")
class ProdPaymentProcessor implements PaymentProcessor { ... }
```

3 Какие реализации IoC можно перечислить?

IoC (Inversion of Control) реализуется через DI. Основные типы:

Тип DI	Описание
Constructor Injection	Зависимость передается через конструктор. Рекомендуется для иммутабельности.
Setter Injection	Зависимость передается через сеттер. Позволяет менять зависимость после создания объекта.
Field Injection	Зависимость внедряется напрямую в поле через <code>@Autowired</code> . Простая, но плохо тестируется.
Interface Injection	Редко используется в Java/Spring. Объект реализует интерфейс, через который контейнер внедряет зависимость.

◆ Ключевой вывод для собеседа

- `@Autowired` → внедрение зависимостей через IoC контейнер
- Несколько реализаций → `@Qualifier`, `@Primary`, `@Profile`
- IoC в Spring реализуется через **constructor**, **setter** и **field injection** (interface injection редко)

1. Какие существуют области видимости (scopes) бинов в Spring и в чем их особенности?
 2. Перечислите этапы поднятия контекста в Spring.
 3. Как можно создать бин в Spring без использования аннотаций?
-

1 Области видимости (scopes) бинов в Spring и их особенности

Spring поддерживает несколько стандартных **scopes**:

Scope	Описание	Особенности
singleton	Один экземпляр на Spring Context	По умолчанию. Все зависимости получают один объект.
prototype	Новый экземпляр каждый раз при запросе	Spring создаёт бин заново при каждом вызове <code>getBean()</code> . Контейнер не управляет lifecycle (init/destroy).
request	Один экземпляр на HTTP-запрос	Только для веб-приложений.
session	Один экземпляр на HTTP-сессию	Веб-приложения, бин живёт пока сессия активна.
application	Один экземпляр на ServletContext	Веб-приложения, общий для всего приложения.
websocket	Один бин на WebSocket-сессию	Для вебсокетов.

◆ Пример объявления scope

```
@Component
@Scope("prototype")
public class MyBean {
}
```

2 Этапы поднятия контекста в Spring

1. **Создание ApplicationContext**
 - Например, `AnnotationConfigApplicationContext` или `ClassPathXmlApplicationContext`
2. **Сканирование бинов (Component Scan)**
 - Контейнер ищет классы с `@Component`, `@Service`, `@Repository`, `@Controller`
3. **Создание и инициализация singleton-бинов**
 - Spring создаёт все singleton-бины
4. **Внедрение зависимостей (DI)**
 - Через конструктор, сеттер или поле (`@Autowired`)
5. **Выполнение методов lifecycle**
 - `@PostConstruct`, `afterPropertiesSet()`
6. **Готовность контекста**
 - Приложение может использовать все бины
7. **Закрытие контекста**
 - `@PreDestroy`, `destroy()` вызываются при закрытии `ApplicationContext`

✚ Для `prototype` scope lifecycle `destroy` не вызывается Spring.

3 Как создать бин в Spring без использования аннотаций?

◆ 1. Через XML-конфигурацию

```
<beans xmlns="http://www.springframework.org/schema/beans"
...>
  <bean id="myBean" class="com.example.MyBean" scope="singleton"/>
</beans>
```

- Контейнер создаст бин при старте (singleton) или по запросу (prototype)
 - Можно внедрять зависимости через `<property>` или `<constructor-arg>`
-

◆ 2. Через Java-конфигурацию (`@Configuration`) — без `@Component`

```
@Configuration
public class AppConfig {

    @Bean
    public MyBean myBean() {
        return new MyBean();
    }
}
```

- `@Bean` создаёт бин вручную
- Можно задать scope через `@Scope("prototype")`
- Подходит для библиотек, где нет аннотаций Spring

◆ 3. Через программное создание бина в ApplicationContext

```
AnnotationConfigApplicationContext context = new  
AnnotationConfigApplicationContext();  
context.registerBean(MyBean.class);  
context.refresh();
```

◆ Ключевые выводы для собеседа

- **Scopes** контролируют lifecycle и видимость бина
- **Контекст Spring** создаётся через несколько этапов: сканирование, создание бинов, DI, lifecycle
- **Бины можно создавать без аннотаций**: XML, Java-конфигурация, программно через context

1. Как в Spring реализуется **Dependency Injection** и какие существуют его типы?
 2. Что такое **Spring Expression Language (SpEL)** и как она используется в **Spring Core**?
 3. Что такое циклические зависимости и как можно решить проблему с циклическими зависимостями?
-

1 Как в Spring реализуется **Dependency Injection (DI)** и какие существуют его типы

Dependency Injection (DI) — это механизм, при котором **Spring контейнер** управляет созданием и внедрением зависимостей между объектами, вместо того чтобы объекты создавали зависимости сами.

◆ Типы DI в Spring

Тип	Описание	Пример
Constructor Injection	Зависимости передаются через конструктор	<pre>java @Component class Service { private final Repo repo; @Autowired public Service(Repo repo) { this.repo = repo; } }</pre>
Setter Injection	Зависимости передаются через сеттер	<pre>java @Component class Service { private Repo repo; @Autowired public void setRepo(Repo repo) { this.repo = repo; } }</pre>
Field Injection	Зависимость внедряется напрямую в поле	<pre>java @Component class Service { @Autowired private Repo repo; }</pre>
Interface Injection	Через интерфейс, который контейнер использует для внедрения (редко в Spring)	Применяется редко, преимущественно теоретически

✦ **Рекомендация:** использовать **constructor injection** для иммутабельности и удобства тестирования.

2 Что такое Spring Expression Language (SpEL) и как она используется

Spring Expression Language (SpEL) — это язык выражений, который позволяет динамически вычислять значения и внедрять их в бины.

◆ Основные возможности SpEL

- Вызов методов
- Доступ к свойствам объектов
- Математические и логические выражения
- Доступ к bean-ам в контексте

◆ Пример использования

```
@Component
public class AppConfig {

    @Value("#{2 * 3}")
    private int result; // result = 6

    @Value("#{systemProperties['user.name']}")
    private String userName;

    @Value("#{myBean.someProperty}")
    private String propertyFromBean;
}
```

- `@Value("#{...}")` позволяет внедрять динамические значения
- Используется в Spring Core для настройки бинов через **XML** или **аннотации**

3 Что такое циклические зависимости и как их решать

Циклическая зависимость — ситуация, когда два или более бина **взаимно зависят друг от друга**, например:

```
@Component
class A {
    @Autowired
    private B b;
}

@Component
class B {
    @Autowired
    private A a;
}
```

◆ Проблемы

- При создании singleton-бинов Spring не сможет их инициализировать напрямую
- Контейнер выбрасывает **BeanCurrentlyInCreationException**

◆ Способы решения

1. Использовать @Lazy

```
@Component
class A {
    @Autowired
    @Lazy
    private B b;
}
```

```
@Component
class B {
    @Autowired
    @Lazy
    private A a;
}
```

- Внедрение происходит **по требованию**, разрывая цикл

2. Constructor Injection + Setter/Field Injection

- Оставить конструктор только для одной зависимости, другую внедрять через @Autowired сеттер

```
@Component
class A {
    private B b;
    @Autowired
    public void setB(B b) { this.b = b; }
}
```

3. Пересмотреть архитектуру

- Часто циклические зависимости — признак плохого дизайна
- Вынести общий функционал в **отдельный бин или сервис**

◆ Ключевые выводы для собеседа

- **DI**: constructor, setter, field (interface редко)
- **SpEL**: динамическое вычисление значений для бинов
- **Циклические зависимости**: @Lazy, смена injection, пересмотр архитектуры

1. Что такое AOP?
 2. Каковы основные принципы работы AOP в Spring?
 3. Какие проблемы решает Spring AOP?
-

1 Что такое AOP?

AOP (Aspect-Oriented Programming, аспектно-ориентированное программирование) — это подход, который **позволяет разделять сквозную функциональность (cross-cutting concerns)** от основной бизнес-логики.

◆ Основная идея

- В обычном коде повторяются однотипные задачи: логирование, транзакции, безопасность, кэширование
- AOP позволяет вынести эти задачи в **отдельные аспекты**, чтобы бизнес-класс оставался “чистым”

Пример:

```
@Service
public class UserService {
    public void createUser(String name) {
        // бизнес-логика
    }
}
```

Логирование через AOP:

```
@Aspect
@Component
public class LoggingAspect {
    @Before("execution(* com.example.UserService.*(..))")
    public void logBefore() {
        System.out.println("Метод вызывается");
    }
}
```

2 Основные принципы работы AOP в Spring

◆ 1. Aspect (Аспект)

- Класс, который содержит сквозную функциональность

◆ 2. Join point (Точка соединения)

- Конкретное место в программе, где может быть применён аспект
- В Spring: вызов метода бина

◆ 3. Advice (Совет / действие)

- Код, который выполняется в определённой точке соединения
- Типы:
 - `@Before` — перед вызовом метода
 - `@After` — после метода (в любом случае)
 - `@AfterReturning` — после успешного выполнения
 - `@AfterThrowing` — при выбрасывании исключения
 - `@Around` — обёртка вокруг метода (до и после)

◆ 4. Pointcut (Срез)

- Выражение, которое определяет, к каким join point применяется advice
- Пример: `execution(* com.example.UserService.*(..))`

◆ 5. Weaving (Встраивание)

- Процесс объединения аспекта с основной логикой
 - В Spring используется **runtime weaving через прокси** (dynamic proxies или CGLIB)
-

3 Какие проблемы решает Spring AOP?

◆ 1. Логирование

- Не дублируется в каждом методе
- Пример: логирование входа и выхода из методов

◆ 2. Транзакции

- `@Transactional` — транзакции управляются аспектом
- Бизнес-логика не “засоряется” кодом транзакций

◆ 3. Безопасность

- Проверка прав доступа через аспекты (`@PreAuthorize`, `@Secured`)

◆ 4. Кэширование

- Применение `@Cacheable`, `@CachePut` без изменения бизнес-кода

◆ 5. Обработка исключений

- Централизованная обработка ошибок через аспект

1. Что такое точка вступления (Join point) в Spring AOP?
 2. Как создать совет (advice) в Spring AOP?
 3. Какие типы советов (advice) поддерживает Spring AOP?
-

1 Что такое точка вступления (Join point) в Spring AOP?

Join point — это конкретное место в программе, где может быть применён аспект (Advice).

- В Spring AOP **join point** — это вызов метода Spring-бина.
- Все остальные действия (например, доступ к полям) не поддерживаются Spring AOP, потому что Spring использует прокси.

◆ Пример

```
@Service
public class UserService {
    public void createUser(String name) { }
}
```

- Вызов `userService.createUser("Alex")` — это **join point**
 - Можно привязать к нему аспект, чтобы выполнить код до или после вызова метода
-

2 Как создать совет (Advice) в Spring AOP?

Advice — это код, который выполняется в точке соединения (join point).

◆ Пример создания аспекта с советом

```
@Aspect
@Component
public class LoggingAspect {

    // Совет, который выполняется перед методом
    @Before("execution(* com.example.UserService.*(..))")
    public void logBefore() {
        System.out.println("Метод вызывается");
    }
}
```

- `@Aspect` — класс-аспект
 - `@Before`, `@After` и т.д. — тип совета
 - `"execution(* com.example.UserService.*(..))"` — **pointcut**, указывающий, к каким методам применять совет
-

3 Какие типы советов (Advice) поддерживает Spring AOP?

Тип совета	Когда выполняется	Пример аннотации
Before	До вызова метода	@Before("pointcut")
After	После метода (в любом случае, независимо от результата)	@After("pointcut")
AfterReturning	После успешного выполнения метода	@AfterReturning("pointcut")
AfterThrowing	При выбросе исключения	@AfterThrowing("pointcut")
Around	Обёртка вокруг метода (до и после)	@Around("pointcut")

◆ Пример `Around`

```
@Around("execution(* com.example.UserService.*(..))")
public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
    System.out.println("До метода");
    Object result = joinPoint.proceed(); // вызов целевого метода
    System.out.println("После метода");
    return result;
}
```

- `ProceedingJoinPoint` нужен только для `@Around`, чтобы вызвать оригинальный метод

◆ Ключевой вывод для собеседования

- **Join point** — место вызова метода, где можно вставить аспект
- **Advice** — код, выполняемый в join point
- **Типы Advice:** `Before`, `After`, `AfterReturning`, `AfterThrowing`, `Around`

1. Как реализовать аспекты с помощью аннотаций `@Aspect` в Spring?
2. Как работает введение (introduction) в Spring AOP?
3. Как настроить порядок выполнения аспектов в Spring AOP?

1 Как реализовать аспекты с помощью аннотаций `@Aspect` в Spring?

`@Aspect` — аннотация, которая указывает, что класс является аспектом.

◆ Шаги реализации аспекта

1. Создать класс и пометить его `@Aspect`

```
@Aspect
@Component
public class LoggingAspect { }
```

2. Определить `pointcut` (точку применения)

```
@Pointcut("execution(* com.example.UserService.*(..))")
public void allUserMethods() {}
```

3. Создать `advice` (совет)

```
@Before("allUserMethods()")
public void logBefore() {
    System.out.println("Метод вызывается");
}
```

4. Регистрация Spring-бина

- Через `@Component` или в конфигурации `@Bean`
- Контейнер создаст прокси и внедрит аспект

◆ Полный пример

```
@Aspect
@Component
public class LoggingAspect {

    @Pointcut("execution(* com.example.UserService.*(..))")
    public void allUserMethods() {}

    @Before("allUserMethods()")
    public void logBefore() {
        System.out.println("До вызова метода UserService");
    }

    @AfterReturning("allUserMethods()")
    public void logAfter() {
        System.out.println("После успешного вызова метода UserService");
    }

}
```

2 Как работает введение (Introduction) в Spring AOP?

Introduction (также известное как **inter-type declaration**) — позволяет добавлять новые методы или интерфейсы существующим бинам без изменения их кода.

- В Spring AOP реализуется через `@DeclareParents`
- Позволяет объекту “приобрести” новый интерфейс

◆ Пример

```
public interface Monitorable {
    void setMonitorActive(boolean active);
    boolean isMonitorActive();
}

@Aspect
@Component
public class MonitoringAspect {

    @DeclareParents(value = "com.example.*", defaultImpl =
DefaultMonitor.class)
    public static Monitorable mixin;
}

public class DefaultMonitor implements Monitorable {
    private boolean active;
    public void setMonitorActive(boolean active) { this.active = active; }
    public boolean isMonitorActive() { return active; }
}
```

- Любой бин в пакете `com.example` теперь “поддерживает” интерфейс `Monitorable`

3 Как настроить порядок выполнения аспектов в Spring AOP?

Порядок выполнения аспектов важен, если несколько аспектов применяются к одному методу.

◆ 1. Аннотация `@Order`

```
@Aspect
@Component
@Order(1)
public class LoggingAspect { ... }

@Aspect
@Component
@Order(2)
public class SecurityAspect { ... }
```

- Меньшее значение `@Order` → выше приоритет → выполняется раньше

◆ 2. Сортировка для `@Around` и других advice

- `@Around` выполняется в соответствии с порядком аспектов, внутренние вызовы `proceed()` формируют стек
-

◆ Ключевой вывод для собеседа

1. **`@Aspect`** → объявление аспекта
2. **Introduction** → добавление методов/интерфейсов существующим бинам через `@DeclareParents`
3. **Порядок выполнения** → через `@Order` (меньшее значение = выше приоритет)

1. Что такое Spring Data?
 2. Каковы основные преимущества использования Spring Data в проектах?
 3. Какие основные интерфейсы предоставляет Spring Data для работы с базами данных?
-

1 Что такое Spring Data?

Spring Data — это проект **Spring**, который упрощает работу с базами данных и системами хранения данных (SQL, NoSQL, поисковые движки и т.д.).

- Абстрагирует реализацию репозитория
- Позволяет писать **минимум кода для CRUD и запросов**
- Поддерживает **JPA, MongoDB, Redis, Cassandra, Elasticsearch** и другие

✦ Ключевая идея:

Интерфейсы репозитория описывают бизнес-запросы, а Spring Data сам их реализует.

2 Основные преимущества использования Spring Data

Преимущество	Описание
Минимум шаблонного кода	CRUD-операции и стандартные запросы реализуются автоматически
Поддержка разных хранилищ	Можно легко менять базу данных, интерфейс репозитория остаётся одинаковым
Query derivation	Создание методов репозитория на основе имени метода (<code>findByNameAndStatus</code>)
Пагинация и сортировка	Встроенные механизмы <code>Pageable</code> и <code>Sort</code>
Интеграция с Spring	Поддержка транзакций, DI, AOP, Spring Boot autoconfiguration
Поддержка кастомных запросов	Через <code>@Query</code> или <code>QueryDSL</code>

3 Основные интерфейсы Spring Data для работы с базами данных

◆ 1. CrudRepository<T, ID>

- Основной интерфейс для CRUD-операций

```
public interface UserRepository extends CrudRepository<User, Long> {}
```

- Методы: `save()`, `findById()`, `findAll()`, `deleteById()`

◆ 2. JpaRepository<T, ID> (расширяет PagingAndSortingRepository)

- Расширяет `CrudRepository`, добавляет:
 - Пагинацию
 - Сортировку
 - Методы `flush()`, `saveAndFlush()`, `deleteInBatch()`

```
public interface UserRepository extends JpaRepository<User, Long> {}
```

◆ 3. PagingAndSortingRepository<T, ID>

- Поддержка пагинации и сортировки

```
Page<User> findAll(Pageable pageable);  
List<User> findAll(Sort sort);
```

◆ 4. MongoRepository, CassandraRepository и др.

- Для NoSQL баз данных

```
public interface ProductRepository extends MongoRepository<Product, String>  
{}
```

◆ 5. QueryDSL и Specification

- Позволяет строить динамические запросы

```
List<User> findAll(Specification<User> spec);
```

◆ Ключевой вывод для собеседа

- **Spring Data** упрощает работу с данными, минимизируя шаблонный код
- Основные интерфейсы: `CrudRepository`, `JpaRepository`, `PagingAndSortingRepository`, `MongoRepository`
- Поддерживает **CRUD**, пагинацию, сортировку, кастомные запросы

1. Как реализовать пагинацию и сортировку в Spring Data?
 2. Как использовать аннотацию `@Query` для создания пользовательских запросов в Spring Data?
 3. Как настроить кэширование запросов в Spring Data?
-

1 Как реализовать пагинацию и сортировку в Spring Data?

Spring Data предоставляет готовые интерфейсы `Pageable` и `Sort`.

◆ Пагинация

```
PageRequest pageRequest = PageRequest.of(0, 10);
Page<User> page = userRepository.findAll(pageRequest);
```

- 0 — номер страницы (начинается с 0)
- 10 — размер страницы

Метод репозитория:

```
Page<User> findAll(Pageable pageable);
```

◆ Сортировка

```
Sort sort = Sort.by(Sort.Direction.DESC, "createdAt");
List<User> users = userRepository.findAll(sort);
```

◆ Пагинация + сортировка

```
Pageable pageable =
    PageRequest.of(0, 10, Sort.by("name").ascending());

Page<User> page = userRepository.findAll(pageable);
```

✦ Обычно `Pageable` приходит из контроллера автоматически:

```
@GetMapping("/users")
public Page<User> getUsers(Pageable pageable) {
    return userRepository.findAll(pageable);
}
```

2 Как использовать аннотацию @Query для пользовательских запросов?

@Query позволяет писать JPQL или SQL-запросы вручную, когда:

- имя метода становится слишком сложным
- нужен нестандартный JOIN
- требуется оптимизация

◆ JPQL-запрос

```
@Query("SELECT u FROM User u WHERE u.status = :status")
List<User> findByStatus(@Param("status") String status);
```

◆ Native SQL-запрос

```
@Query(
    value = "SELECT * FROM users WHERE active = true",
    nativeQuery = true
)
List<User> findActiveUsers();
```

◆ @Query + пагинация

```
@Query("SELECT u FROM User u WHERE u.age > :age")
Page<User> findOlderThan(@Param("age") int age, Pageable pageable);
```

✦ Spring Data сам добавит LIMIT / OFFSET.

3 Как настроить кэширование запросов в Spring Data?

Spring Data использует Spring Cache Abstraction.

◆ Шаг 1. Включить кэширование

```
@EnableCaching
@Configuration
public class CacheConfig {}
```

◆ Шаг 2. Добавить кэш к методу репозитория или сервиса

```
@Cacheable("users")
public User getUserById(Long id) {
    return userRepository.findById(id).orElseThrow();
}
```

- Первый вызов → запрос в БД
- Следующие вызовы → данные из кэша

◆ Очистка кэша

```
@CacheEvict(value = "users", key = "#id")
public void deleteUser(Long id) {
    userRepository.deleteById(id);
}
```

◆ Обновление кэша

```
@CachePut(value = "users", key = "#result.id")
public User updateUser(User user) {
    return userRepository.save(user);
}
```

✚ Поддерживаются разные провайдеры:

- EhCache
- Caffeine
- Redis
- Hazelcast

◆ Ключевые выводы для собеседования

- Пагинация и сортировка → Pageable, Sort
- Кастомные запросы → @Query (JPQL / native SQL)
- Кэширование → Spring Cache (@Cacheable, @CacheEvict, @CachePut)

1. Как использовать Spring Data REST для создания RESTful сервисов?
 2. Какие стратегии оптимистичного и пессимистичного блокирования поддерживает Spring Data JPA, и как их применять?
 3. Как настроить аудит сущностей в Spring Data JPA?
-

1 Как использовать Spring Data REST для создания RESTful сервисов?

Spring Data REST — это надстройка над Spring Data, которая автоматически публикует CRUD-репозитории как REST-эндпоинты, без написания контроллеров.

◆ Подключение

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
```

◆ Репозиторий

```
@RepositoryRestResource(path = "users")
public interface UserRepository extends JpaRepository<User, Long> {
}
```

◆ Автоматически доступны эндпоинты

HTTP	URL	Описание
GET	/users	получить список
GET	/users/{id}	получить по id
POST	/users	создать
PUT/PATCH	/users/{id}	обновить
DELETE	/users/{id}	удалить

✚ Ответы идут в **HAL/JSON** с ссылками (`_links`).

◆ Ограничение экспозиции

```
@RepositoryRestResource(exported = false)
public interface InternalRepository extends JpaRepository<...> {}
```

✚ Когда использовать:

- быстрые прототипы
- админ-панели
- внутренние сервисы

✦ Когда не стоит:

- сложная бизнес-логика
- строгий REST-контракт
- публичные API

2 Оптимистичное и пессимистичное блокирование в Spring Data JPA

◆ Оптимистичное блокирование (Optimistic Locking)

Используется, когда **конфликты редки**.

Принцип:

- В сущности есть поле версии
- При обновлении проверяется версия
- Если версия изменилась → `OptimisticLockException`

```
@Entity
public class Order {

    @Id
    private Long id;

    @Version
    private Long version;
}
```

✦ Используется по умолчанию, рекомендуемый вариант.

◆ Пессимистичное блокирование (Pessimistic Locking)

Используется, когда **конфликты часты**.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT o FROM Order o WHERE o.id = :id")
Optional<Order> findByIdForUpdate(@Param("id") Long id);
```

Типы:

Тип	Описание
<code>PESSIMISTIC_READ</code>	блокирует чтение
<code>PESSIMISTIC_WRITE</code>	блокирует чтение и запись

✦ Работает через SQL-блокировки (`SELECT FOR UPDATE`)

◆ Сравнение

Критерий	Optimistic	Pessimistic
Производительность	высокая	ниже
Блокировки БД	нет	есть
Подходит для	web-приложений критических транзакций	

3 Как настроить аудит сущностей в Spring Data JPA?

Аудит — автоматическое заполнение полей:

- кто создал
 - когда создано
 - кто обновил
 - когда обновлено
-

◆ Шаг 1. Включить аудит

```
@EnableJpaAuditing
@Configuration
public class JpaConfig {
}
```

◆ Шаг 2. Аудируемая сущность

```
@Entity
@EntityListeners(AuditingEntityListener.class)
public class User {

    @CreatedDate
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String updatedBy;
}
```

◆ Шаг 3. AuditorAware

```
@Component
public class AuditorAwareImpl implements AuditorAware<String> {

    @Override
    public Optional<String> getCurrentAuditor() {
        return Optional.of("system");
    }
}
```

✦ Обычно берётся из **Spring Security Context**.

◆ Итог для собеседования

- **Spring Data REST** → авто-REST поверх репозиторий
 - **Optimistic locking** → @Version
 - **Pessimistic locking** → @Lock
 - **Аудит** → @CreatedDate, @LastModifiedDate, AuditorAware
-

1. Что такое Spring MVC?
 2. Какова основная роль аннотации `@Controller` в Spring MVC?
 3. Какие шаги необходимо выполнить для создания простого веб-приложения на Spring MVC?
-

1 Что такое Spring MVC?

Spring MVC — это веб-фреймворк в составе Spring Framework, реализующий архитектурный паттерн **Model–View–Controller (MVC)**.

◆ Идея MVC

- **Model** — данные и бизнес-логика
- **View** — представление (HTML, JSON, JSP, Thymeleaf и т.п.)
- **Controller** — принимает HTTP-запросы, обрабатывает их и возвращает ответ

◆ Что делает Spring MVC

- принимает HTTP-запросы
- маппит их на методы контроллеров
- связывает параметры запроса с объектами
- обрабатывает валидацию
- формирует HTTP-ответ (HTML или JSON)

✚ В современных приложениях Spring MVC чаще используется как основа **REST API**.

2 Какова основная роль аннотации `@Controller` в Spring MVC?

`@Controller` помечает класс как **веб-контроллер**, который:

- обрабатывает HTTP-запросы
- возвращает **имя представления (View)**

```
@Controller
public class HelloController {

    @GetMapping("/hello")
    public String hello(Model model) {
        model.addAttribute("message", "Hello Spring MVC");
        return "hello"; // имя view
    }
}
```

♦ Важно различать

Аннотация	Назначение
@Controller	Возвращает View (HTML, JSP)
@RestController	Возвращает данные (JSON/XML)
@RestController	
<pre>public class ApiController { @GetMapping("/api/hello") public String hello() { return "Hello REST"; } }</pre>	
 @RestController = @Controller + @ResponseBody	

3 Шаги для создания простого веб-приложения на Spring MVC

♦ Шаг 1. Создать Spring Boot проект

Зависимости:

- spring-boot-starter-web
 - (опционально) spring-boot-starter-thymeleaf
-

♦ Шаг 2. Создать контроллер

```
@Controller  
public class HomeController {  
  
    @GetMapping("/")  
    public String home(Model model) {  
        model.addAttribute("text", "Welcome to Spring MVC");  
        return "home";  
    }  
}
```

♦ Шаг 3. Создать View (например, Thymeleaf)

src/main/resources/templates/home.html

```
<!DOCTYPE html>  
<html>  
<body>  
<h1 th:text="${text}"></h1>  
</body>  
</html>
```

◆ Шаг 4. Запустить приложение

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Открыть в браузере:

`http://localhost:8080/`

◆ (Опционально) REST-вариант

```
@RestController
@RequestMapping("/api")
public class UserController {

    @GetMapping("/users")
    public List<String> users() {
        return List.of("Alex", "John");
    }
}
```

◆ Итог для собеседования

- **Spring MVC** — веб-фреймворк на базе паттерна MVC
- **@Controller** — обрабатывает HTTP-запросы и возвращает View
- **Основные шаги:**
 1. зависимости
 2. контроллер
 3. view или REST-ответ
 4. запуск приложения

1. Как в Spring MVC реализуется обработка запросов на стороне сервера?
2. Какие преимущества предоставляет использование `ViewResolver` в Spring MVC?
3. Как можно обработать исключения в приложении Spring MVC?

Отвечаю чётко, системно и с примерами, как на собеседовании.

□ Как в Spring MVC реализуется обработка запросов на стороне сервера?

Обработка запроса в Spring MVC построена вокруг `DispatcherServlet` — центрального фронт-контроллера.

◆ Поток обработки HTTP-запроса

1. Клиент отправляет HTTP-запрос
2. `DispatcherServlet` принимает запрос
3. **HandlerMapping**
 - находит подходящий контроллер и метод (`@RequestMapping`, `@GetMapping`)
4. **HandlerAdapter**
 - вызывает метод контроллера
5. **Controller**
 - обрабатывает запрос
 - возвращает:
 - `ModelAndView`
 - имя view (`String`)
 - или тело ответа (`@ResponseBody`)
6. **ViewResolver**
 - определяет, какое представление использовать
7. **View**
 - рендерит HTML/JSON
8. Ответ возвращается клиенту

◆ Схема (словами)

```
Request → DispatcherServlet
        → HandlerMapping
        → Controller
        → ViewResolver
        → View
        → Response
```

✦ В REST-контроллерах этап `ViewResolver` пропускается — данные сериализуются сразу в JSON.

❏ Какие преимущества даёт использование ViewResolver в Spring MVC?

ViewResolver — это компонент, который по имени **View** определяет конкретное представление.

◆ Пример

Контроллер:

```
@GetMapping("/home")
public String home() {
    return "index";
}
```

ViewResolver:

```
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

→ Spring подставит:

```
/WEB-INF/views/index.jsp
```

◆ Преимущества

Преимущество	Объяснение
Разделение логики и представления	Контроллер не знает, как рендерится View
Гибкость	Легко сменить JSP → Thymeleaf
Централизованная настройка	Один resolver для всех контроллеров
Поддержка разных View	JSP, Thymeleaf, FreeMarker, PDF

✦ В Spring Boot ViewResolver настраивается **автоматически**.

❏ Как можно обработать исключения в Spring MVC?

Spring MVC предоставляет несколько уровней обработки исключений.

◆ 1. Локальная обработка через @ExceptionHandler

```
@Controller
public class UserController {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handle(UserNotFoundException ex) {
        return ResponseEntity
```

```
        .status(HttpStatus.NOT_FOUND)
        .body(ex.getMessage());
    }
}
```

✦ Работает только в этом контроллере.

◆ 2. Глобальная обработка через @ControllerAdvice

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleAll(Exception ex) {
        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Ошибка сервера");
    }
}
```

✦ Рекомендуемый способ для REST API.

◆ 3. Аннотация @ResponseStatus

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class UserNotFoundException extends RuntimeException {
}
```

✦ Простая и декларативная обработка.

◆ 4. HandlerExceptionResolver

- Низкоуровневый механизм
 - Используется редко, для кастомной логики
-

◆ Итог для собеседования

- Обработка запросов → DispatcherServlet + цепочка компонентов
- ViewResolver → связывает имя View с реальным шаблоном
- Обработка исключений:
 - @ExceptionHandler
 - @ControllerAdvice
 - @ResponseStatus

1. Как реализовать международную поддержку (i18n) в приложении Spring MVC?
 2. Какие стратегии можно применить для оптимизации производительности веб-приложения на Spring MVC?
 3. Как в Spring MVC реализовать аутентификацию и авторизацию с использованием Spring Security?
-

1 Как реализовать международную поддержку (i18n) в Spring MVC?

i18n (internationalization) в Spring MVC строится вокруг:

- MessageSource
 - LocaleResolver
 - LocaleChangeInterceptor
-

◆ Шаг 1. Файлы сообщений

```
messages.properties
messages_en.properties
messages_ru.properties
# messages_ru.properties
welcome=Добро пожаловать
# messages_en.properties
welcome=Welcome
```

◆ Шаг 2. MessageSource

```
@Bean
public MessageSource messageSource() {
    ReloadableResourceBundleMessageSource source =
        new ReloadableResourceBundleMessageSource();
    source.setBasename("classpath:messages");
    source.setDefaultEncoding("UTF-8");
    return source;
}
```

(Spring Boot — автоконфигурация, часто достаточно
spring.messages.basename=messages)

◆ Шаг 3. LocaleResolver

```
@Bean
public LocaleResolver localeResolver() {
    SessionLocaleResolver resolver = new SessionLocaleResolver();
    resolver.setDefaultLocale(Locale.ENGLISH);
    return resolver;
}
```

Варианты:

- SessionLocaleResolver
 - CookieLocaleResolver
 - AcceptHeaderLocaleResolver
-

◆ Шаг 4. Смена языка

```
@Bean
public LocaleChangeInterceptor localeChangeInterceptor() {
    LocaleChangeInterceptor interceptor =
        new LocaleChangeInterceptor();
    interceptor.setParamName("lang");
    return interceptor;
}

@Override
public void addInterceptors(InterceptorRegistry registry) {
    registry.addInterceptor(localeChangeInterceptor());
}
```

URL:

/home?lang=ru

◆ Использование во View (Thymeleaf)

```
<h1 th:text="#{welcome}"></h1>
```

2 Стратегии оптимизации производительности Spring MVC

◆ 1. Асинхронная обработка

```
@GetMapping("/async")
public Callable<String> async() {
    return () -> "result";
}
```

или

```
@GetMapping("/deferred")
public DeferredResult<String> deferred() {
    DeferredResult<String> result = new DeferredResult<>();
    // async processing
    return result;
}
```

◆ 2. Кэширование

- @Cacheable для сервисов
- HTTP-кэширование (ETag, Cache-Control)

```
@Cacheable("users")
public User getUser(Long id) { ... }
```

◆ 3. Оптимизация сериализации

- Jackson tuning
 - DTO вместо Entity
 - Исключение лишних полей (@JsonIgnore)
-

◆ 4. Снижение нагрузки

- Pagination (Pageable)
 - Lazy loading
 - Connection pool (HikariCP)
-

◆ 5. Статические ресурсы

- CDN
- кэширование ресурсов

```
spring.web.resources.cache.period=365d
```

3 Аутентификация и авторизация в Spring MVC (Spring Security)

◆ Основные понятия

- **Authentication** — кто ты?
 - **Authorization** — что тебе можно?
-

◆ Подключение

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

◆ Конфигурация безопасности

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {

        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(Customizer.withDefaults())
            .httpBasic();

        return http.build();
    }
}
```

◆ Аутентификация

- In-memory
- JDBC
- LDAP
- JWT / OAuth2

```
@Bean
public UserDetailsService users() {
    return new InMemoryUserDetailsManager(
        User.withUsername("user")
            .password("{noop}123")
            .roles("USER")
            .build()
    );
}
```

◆ Авторизация на уровне методов

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) {}
```

◆ Итог для собеседования

- **i18n** → MessageSource + LocaleResolver
 - **Производительность** → async, cache, pagination, pooling
 - **Security** → фильтры, SecurityFilterChain, роли, аннотации
-